

9 数据库并发控制与封锁

01

OPTION

数据库并发性的含义

02

OPTION

事务及其性质

03

OPTION

并发控制与数据的不一致性

04

OPTION

事物的隔离级别

05

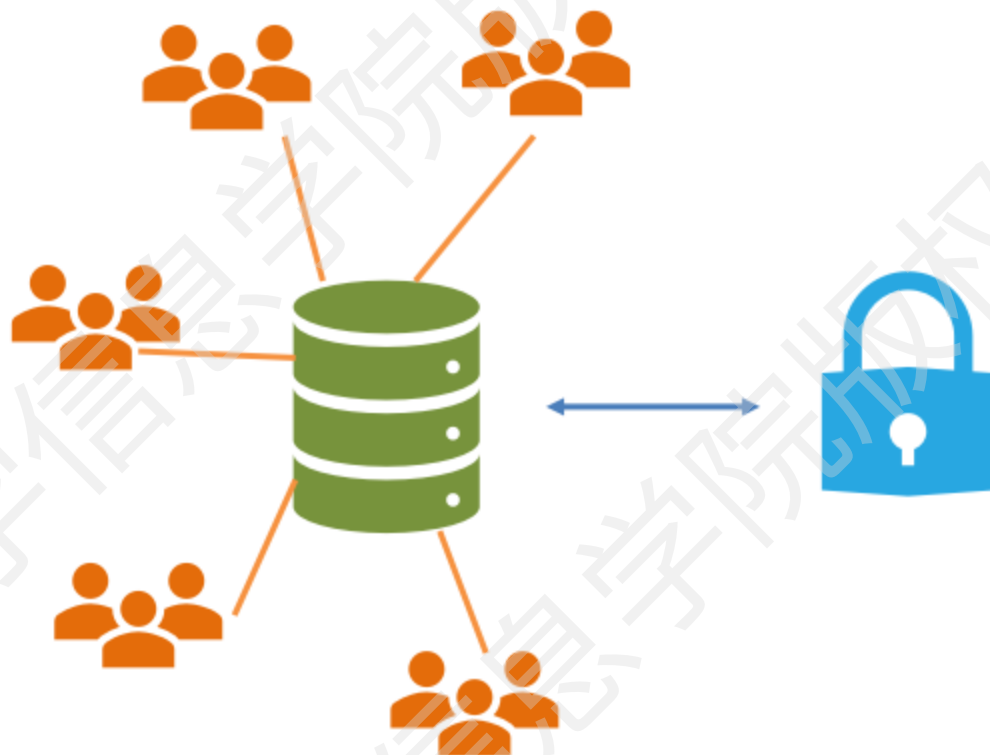
OPTION

封锁及封锁协议

06

OPTION

总结



数据库最大的特点是**数据共享**，允许**同一时间**供**多个用户**根据自己的需要来操作数据库。

串行 一个用户在运行程序时，其他用户程序必须等到这个用户程序结束才能对数据库进行存取。
如果某个用户程序涉及大量数据的输入、输出操作，那么大部分时间将处于闲置状态。

并行 最大限度地利用数据库，多个用户并行执行数据库。
充分提高系统的执行效率。

？ 数据的不一致性

数据库的**并发性及并发控制机制**是衡量数据库系统性能的重要标准。

● 事务的概念

事务是实现数据库中数据一致性的重要技术。数据库事务是一个**不可分割**的逻辑工作单元。对其概念的理解需要注意以下几点：

- 事务中包含的操作可以是一个，也可是多个，但这些操作必须构成一个**逻辑上的整体**。
- 构成事务的所有操作，要么全都对数据库产生影响，要么全都不产生影响，即不管事务是否执行成功，用户看到的数据总能**保持一致性**。
- 事务执行的结果是使数据库**从一种一致性状态转变到另一种一致性状态**。
- 以上即使在数据库出现故障或并发事务存在的情况下**仍然成立**。

- 事务的概念

MySQL支持4种事务模式:

自动提交模式

显式模式

隐式模式

分布式模式

用户定义的模式

适合多服务器系统

- 事务的概念

MySQL中对事务的管理的操作包括启动、结束和回滚，常用的语法格式：

START TRANSACTION

事务启动

COMMIT

ROLLBACK

提交所执行的所有操作，标志事务的结束

回滚语句，当事务运行过程中发生故障时，事务不能继续执行，此时回滚操作撤销了事务所做的修改，并结束当前这个事务

事务的性质

A

原子性 (Atomicity)

C

一致性 (Consistency)

I

隔离性 (Isolation)

D

持久性 (Durability)



A1在银行的账户: account1

A2在银行的账户: account2

← 客户A1向客户A2转账R元



转账事务的操作与语句

● 转账事务的操作与语句

在MySQL环境中，转账事务的语句为：



A1在银行的账户：account1

A2在银行的账户：account2

← 客户A1向客户A2转账100元



(1) 读取账户account1的余额;

(2) 从账户account1中转出相应金额;

(3) 把结果写回account1中;

(4) 读取账户account2的余额;

(5) 向账户account2中转入相应金额;

(6) 把结果写回account2中。

```
use bankaccount;
```

```
start transaction;
```

```
select * from account;
```

```
select R from account where id='account1';
```

```
update account set R=R-100 where id='account1';
```

```
update account set datetime=now() where id='account1';
```

```
select R from account where id='account2';
```

```
update account set R=R+100 where id='account2';
```

```
update account set datetime=now() where id='account2';
```

```
commit;
```

● 事务的性质

原子性 (Atomicity) 意指事务中的所有操作作为一个整体像原子一样不可分割。

A

原子性 (Atomicity)

- 操作要么都执行，要么都不执行
- 一旦在执行某一步骤的过程中发生故障，就需要执行回滚操作，撤销前述操作

```
use bankaccount;
start transaction;
select * from account;
select R from account where id='account1';
update account set R=R-100 where id='account1';
update account set datetime=now() where id='account1';
select R from account where id='account2';
update account set R=R+100 where id='account2';
update account set datetime=now() where id='account2';
commit;
```

● 事务的性质

一致性 (Consistency) 指数据库始终保持一致性。

C

一致性 (Consistency)

- 事务的执行结果必须使数据库从一个一致性状态到另一个一致性状态
- 执行事务之前和执行之后, account1和account2的总金额保持一致

```
use bankaccount;
start transaction;
select * from account;
select R from account where id='account1';
update account set R=R-100 where id='account1';
update account set datetime=now() where id='account1';
select R from account where id='account2';
update account set R=R+100 where id='account2';
update account set datetime=now() where id='account2';
commit;
```

● 事务的性质

隔离性 (Isolation) 指并发执行的事务之间不会相互影响。

隔离性 (Isolation)

- 事务未提交 (COMMIT) 时, account1和account2账户的金额都不会发生变化
- 如果同时account3向account2转账, 当两个事务都执行结束的时候, 结果应与两个事务分别执行的结果一致。

```
use bankaccount;
start transaction;
select * from account;
select R from account where id='account1';
update account set R=R-100 where id='account1';
update account set datetime=now() where id='account1';
select R from account where id='account2';
update account set R=R+100 where id='account2';
update account set datetime=now() where id='account2';
commit;
```

● 事务的性质

持久性 (Durability) 指一个事务被提交后，对数据库中数据的改变是永久性的。

D

持久性 (Durability)

- 转账成功，两个账户里的金额就会写入数据库做持久化保存
- 数据库系统遇到故障的情况下也不会丢失提交事务的操作

```
use bankaccount;  
start transaction;  
select * from account;  
select R from account where id='account1';  
update account set R=R-100 where id='account1';  
update account set datetime=now() where id='account1';  
select R from account where id='account2';  
update account set R=R+100 where id='account2';  
update account set datetime=now() where id='account2';  
commit;
```

并发操作

多个用户同时访问同一个数据资源并对其进行操作时，称之为并发操作。

问题

并发操作时，如果不加以适当控制，可能会存储不正确的数据，产生数据的不一致性问题。

并发控制

能够合理调度并发事务，避免并发事务之间的互相干扰造成数据不一致性的技术称为数据库管理系统的并发控制技术。

● 数据的不一致性的种类

A

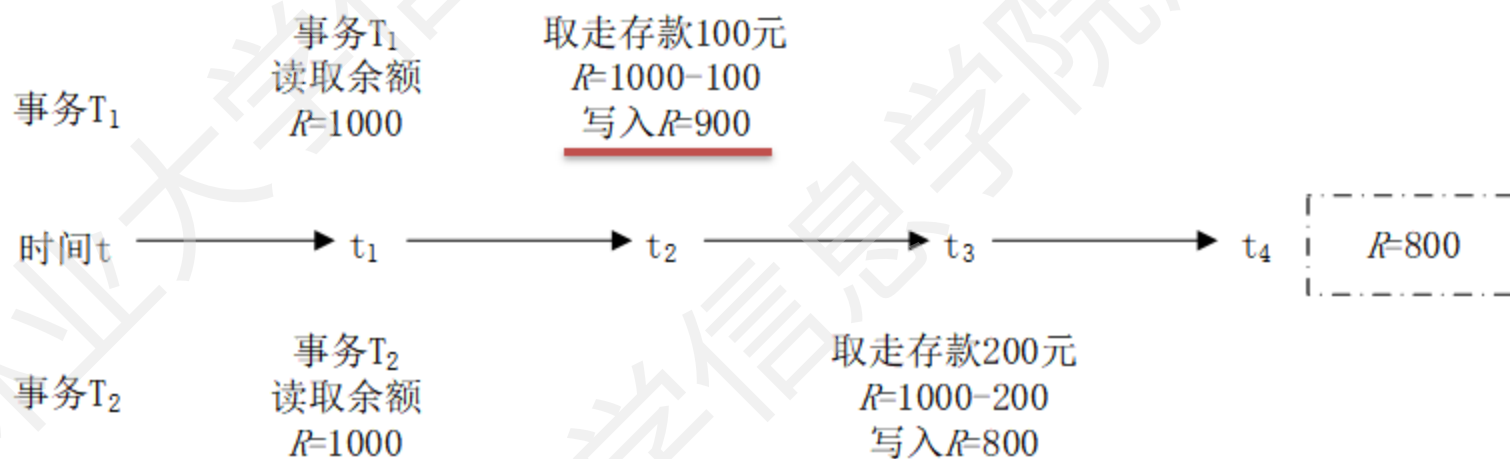
OPTION

丢失更新

Lost Update

并发执行修改操作时，一个事务把另一个对数据的已修改结果覆盖，导致这些修改好像丢失了一样，从而造成了数据的不一致，这种并发性问题称之为丢失更新（Lost Update）。

- (1) 事务T1读取余额1000元;
- (2) 事务T2读取余额1000元;
- (3) 事务T1取走存款100元;
- (4) 事务T2取走存款200元。



● 数据的不一致性的种类

B
OPTION

读取脏数据

Dirty Read

一个事务读取了另一个事务未提交的数据，会造成两个事务得到的数据不一致，称之为读取脏数据，也称为“脏读”（Dirty Read）。

- (1) 事务T1在t1时刻读取了R=1000;
- (2) 事务T1修改R=900，但尚未提交;
- (3) 事务T2读取了T1尚未提交的R;
- (4) 事务T1回滚，R的值恢复为1000;
- (5) 事务T2依然在使用读出的900。

时 间	事务T1	数据库中R的值	事务T2
t0		1000	
t1	SELECT R		
t2	R=R-100		
t3	UPDATE R		
t4		900	SELECT R
t5	ROLLBACK		
t6		1000	

● 数据的不一致性的种类

C

OPTION

不可重复读

Unrepeatable Read

一个事务对同一数据的读取结果前后不一致，这是由于在两次查询期间该数据被另一个事务修改并提交了。当其中一个事务需要校验或再确认数据时，出现再读数据与之前读的数据值不相同。

- (1) 事务T1在t1时刻读取了R=1000;
- (2) 事务T2在t2时刻读取了R=1000;
- (3) 事务T1修改R=900;
- (4) 事务T2修改R为800;
- (5) 事务T1再次读取进行核验, R=800。

时 间	事务T1	数据库中R的值	事务T2
t0		1000	
t1	SELECT R		
t2			SELECT R
t3	R=R-100		
t4	<u>UPDATE R</u>		
t5		900	
t6			R=R-200
t7			UPDATE R
t8		800	
t9	<u>SELECT R</u>		

● 数据的不一致性的种类



OPTION

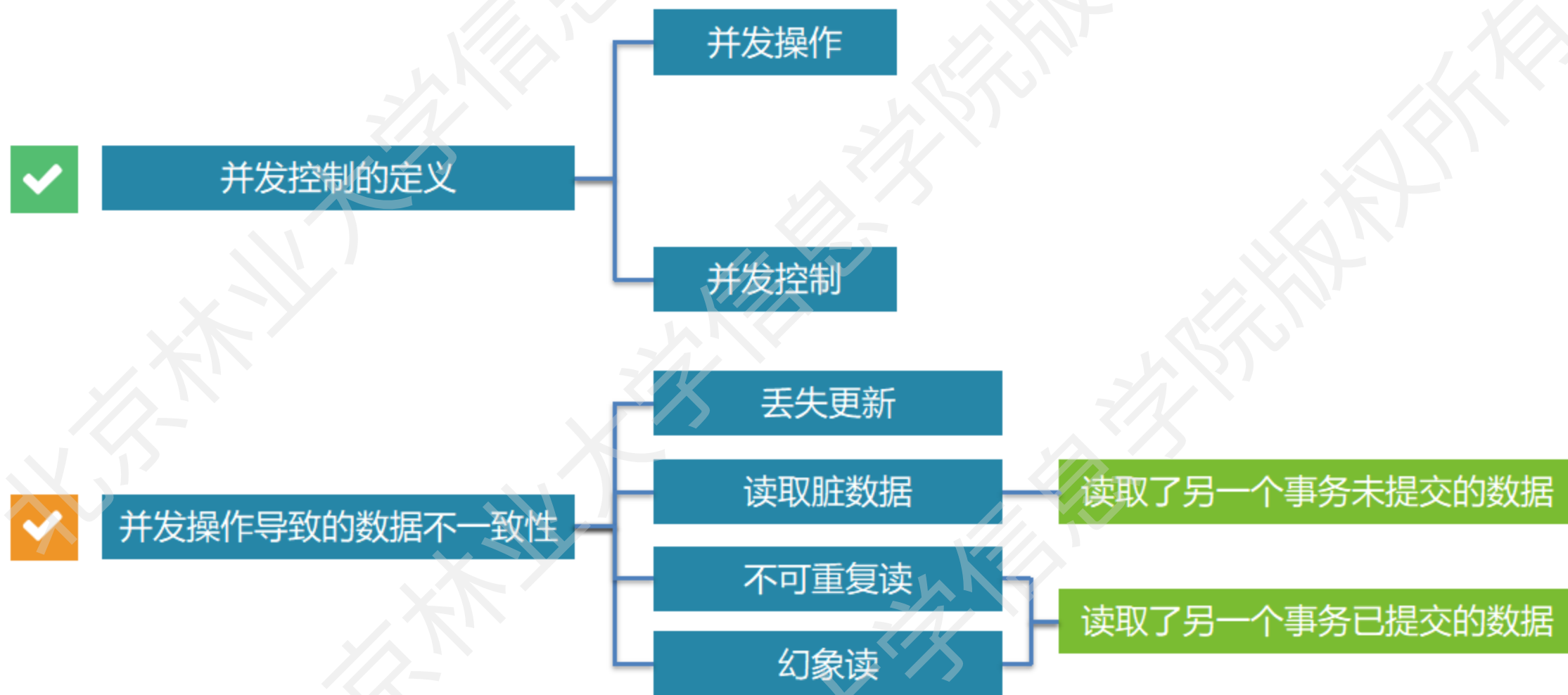
幻象读

Phantom Read

当用相同的条件查询记录时，由于有并发事务在对相同的表做插入或删除操作，导致记录个数忽多忽少，有一种“幻象”的感觉。

- (1) 事务T1在t1时刻查询得到3行记录；
- (2) 事务T2插入一条记录；
- (3) 事务T1在t5时刻以相同条件查询时，得到4行记录。

时 间	事务T1	数据表的记录数	事务T2
t0		<u>3</u>	
t1	SELECT (R<1000)		
t2			WRITE (R)=800
t3			COMMIT
t4		<u>4</u>	
t5	SELECT (R<1000)		



产生上述四种数据不一致性问题的主要原因是并发的事务操作**破坏了事务的隔离性**。为了防止数据库的并发操作导致的数据库不一致性问题，SQL标准定义了4种隔离级别：

读取未提交的数据 (READ UNCOMMITTED)

读取提交的数据 (READ COMMITTED)

可重复读 (REPEATABLE READ)

串行化 (SERIALIZABLE)

低 — 可能出现的并发异常多

高

MySQL数据库支持所有的隔离级别，查询当前事务隔离级别的语句：

```
SELECT @@transaction_isolation;
```

设置当前事务的隔离级别有两种方式：

SET [global | session] transaction isolation level 隔离级别名称;

SET transaction_isolation='隔离级别名称';

隔离级只适用于当前运行的会话和连接

定义的隔离级适用于所有的用户

读取未提交的数据 (READ UNCOMMITTED)

读取未提交数据READ UNCOMMITTED是**最低**事务隔离级别。设置该级别的语法格式如下：

```
SET [GLOBAL | SESSION] TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
```

读取未提交的数据 (READ UNCOMMITTED)

【例9-2】 设置事务T₁的隔离级别为读取未提交的数据。事务T₂向余额存款100元，在T₂尚未提交结果时，事务T₁读取余额。

T₁:

-- 设置T₁的隔离级别

set session transaction isolation level read uncommitted;

-- 开启事务

start transaction;

-- 第一次查询，之后转到事务T₂

select * from account;

T₂:

-- 开启事务

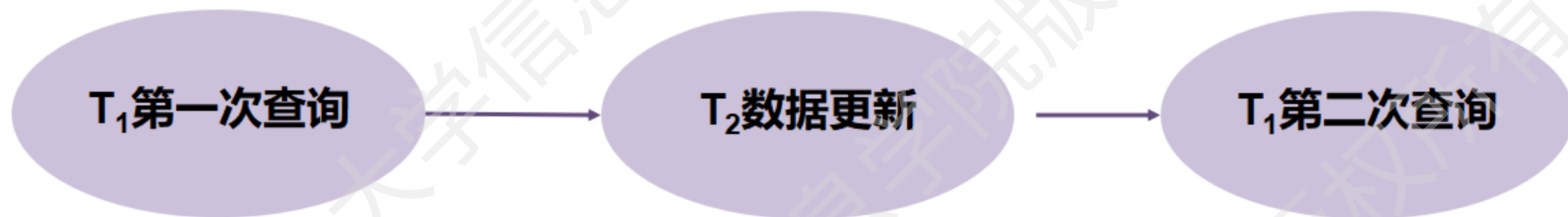
start transaction;

-- 不提交，转到事务T₁

update account set R=R+100 where id="account1";

id	R
account3	900
account1	500
account2	900

读取未提交的数据 (READ UNCOMMITTED)



T₁:

-- 第二次查询，发现两次结果不一致，读取了事务T₂未提交的数据，发生了脏读

select * from account;

id	R
account1	600
account2	900
account3	900

读取未提交 (READ UNCOMMITTED) 的隔离级别最低，**无法避免读取脏数据、不可重复读和幻象读。**

读取提交的数据 (READ COMMITTED)

读取提交的数据比上一级别稍高，该级别下的事务只能读取其他事务已经提交的数据，满足了隔离性的简单定义。设置该级别的语法格式如下：

```
SET [GLOBAL | SESSION] TRANSACTION ISOLATION LEVEL READ COMMITTED
```

读取提交的数据 (READ COMMITTED)

【例9-3】设置事务T1的隔离级别为读取提交的数据Read Committed。事务T1和事务T2同时读取账户余额；紧接着事务T2向账户存入100元不提交，然后事务T1再次读取账户余额，发现账户未发生改变，不读取未提交的数据；之后事务T2提交，最后事务T1第三次读取账户余额，账户多了100元，读取事务T2提交的数据。

T₁:

-- 设置T₁的隔离级别

set session transaction isolation level read committed;

-- 开启事务

start transaction;

-- 第一次查询，之后转到account事务T₂

select * from account;

id	R
account1	500
account2	900
account3	900

读取提交的数据 (READ COMMITTED)

事务 T_2 :

-- 开启事务

start transaction;

-- 不提交, 转到事务 T_1

update account set $R=R+100$ where id="account1";

事务 T_1 :

-- 第二次查询, 结果和第一次查询一样, 没有读取事务 T_2 未提交的数据, 之后转到事务 T_2

select * from account;

事务 T_2 :

-- 提交, 转到事务 T_1

commit;

事务 T_1 :

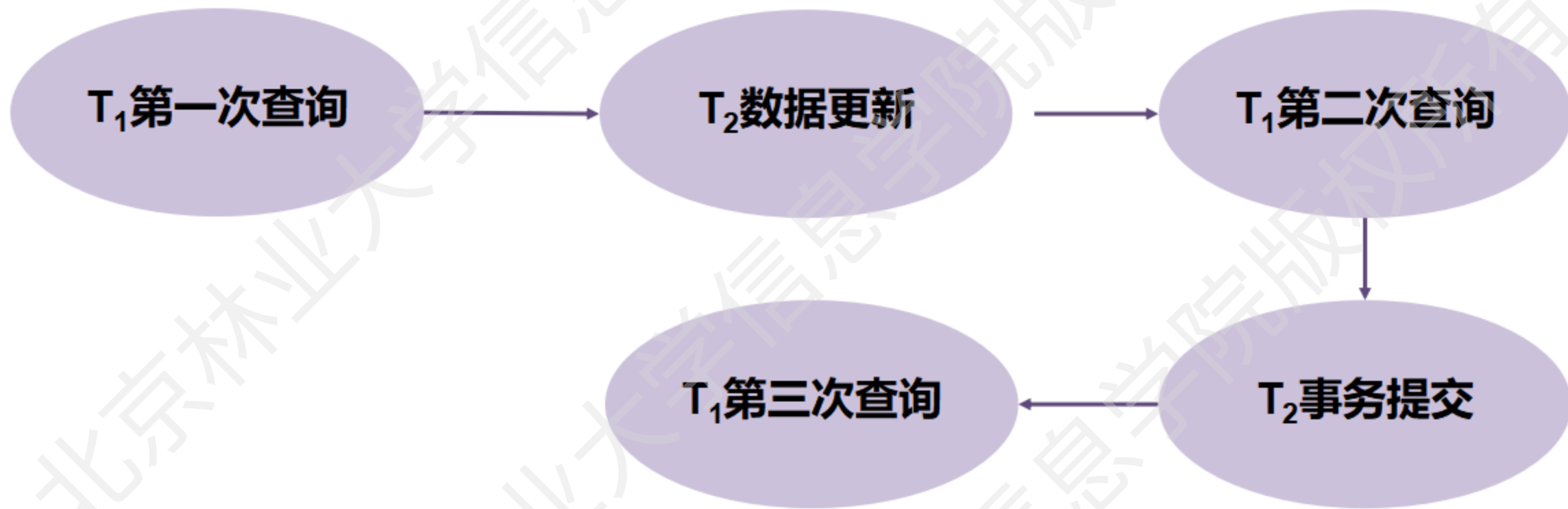
-- 第三次查询, 发现账户余额多了100元, 读取了事务 T_2 提交的数据, 和前两次读取的余额是不同的结果 (不可重复读)

select * from account;

id	R
account1	500
account2	900
account3	900

id	R
account1	600
account2	900
account3	900

读取提交的数据 (READ COMMITTED)



当把事务的隔离级别设置为READ COMMITTED时，可以避免读取脏数据，但**不可避免不可重复读和幻象读**。

可重复读 (REPEATABLE READ)

可重复读REPEATABLE READ是**MySQL默认**的隔离级别，确保同一事务内执行相同的查询语句时，读取的结果是一致的。设置该级别的语法格式如下：

```
SET [GLOBAL | SESSION] TRANSACTION ISOLATION LEVEL REPEATABLE READ
```

可重复读 (REPEATABLE READ)

【例9-4】设置事务T₁的隔离级别为可重复读。事务T₁读取表中的数据，之后事务T₂更新account并提交，然后T₁读取表中的数据。

T₁:

-- 设置T₁的隔离级别

set session transaction isolation level repeatable read;

-- 开启事务

start transaction;

T₂: -- 第一次查询，之后转到account事务T₂

-- 开启事务

start transaction;

-- 修改账户中的余额

update account set R=R+100 where id="account1";

-- 事务T₂内读取结果，余额发生了变化，修改成功

select * from account;

id	R
account1	500
account2	900
account3	900

-- 提交，转到事务T₁

commit;

id	R
account1	600
account2	900
account3	900

可重复读 (REPEATABLE READ)

T_1 :

-- 第二次查询，两次读取的余额是相同的结果，避免了不可重复读
select * from account;

id	R
account1	500
account2	900
account3	900

T_1 第一次查询

T_2 数据更新

T_2 事务提交

T_1 第二次查询

可重复读隔离级别有效避免了不可重复读的问题，但仅是针对同行数据而言，如果事务 T_2 对多行数据进行增加，那么将会出现**幻象读**的问题。

串行化 (SERIALIZABLE)

串行化SERIALIZABLE的隔离级别最高，通过强制事务排序，使事务之间不可能相互冲突。设置该级别的语法格式如下：

```
SET [GLOBAL | SESSION] TRANSACTION ISOLATION LEVEL SERIALIZABLE
```

用户之间一个接一个顺序地执行当前事务，从而解决幻象读问题，但是可能导致大量的等待现象。

事务隔离级别与能够避免的并发异常对应情况

隔离级别	丢失更新	读取脏数据	不可重复读	幻象读
读取未提交的数据 (READ UNCOMMITTED)				
读取提交的数据 (READ COMMITTED)		√		
可重复读 (REPEATABLE READ)	√	√	√	
串行化 (SERIALIZABLE)	√	√	√	√

- 隔离级别的选择对于每个应用程序是没有标准的，需要**基于不同的事务选择不同的级别**。
- 事务隔离性的实现按照其**对可能重读的操作采取的不同策略**可以分为乐观并发控制和悲观并发控制两大类。
- **乐观并发控制**：对于并发执行可能冲突的操作，假定其不会真的冲突，允许并发执行，直到真正发生冲突时才去解决冲突，比如让事务回滚。
- **悲观并发控制**：对于并发执行可能冲突的操作，假定其必定发生冲突。

封锁是一种用来防止多个事务同时访问数据而产生问题的机制。

封锁可以防止用户读取正在由其他用户修改的数据，并可以防止多个用户同时更改相同数据。

事务T在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其加锁。加锁后事务T对该数据对象就有了一定的控制，在事务T释放它的锁之前，其他事务不能更新该数据。

- 封锁粒度

封锁的数据库对象的大小称为封锁粒度。

锁定的数据量越少，发生锁争用的可能就越小，系统的并发程度就越高。

实际使用时，需要综合考虑锁开销和并发程度，对系统的开销与并发度进行权衡，选择合适的封锁粒度。

MySQL提供了两种封锁粒度

行级锁

表级锁

● 封锁粒度

表级锁

整个表被锁定

开销小，加锁快
不会出现死锁

锁定粒度大，发生锁冲突
的概率最高，并发度最低

行级锁

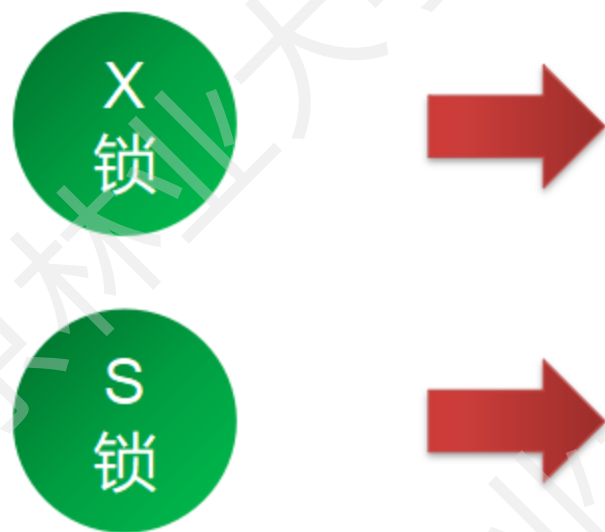
正在使用的行
被锁定

开销大，加锁慢
会出现死锁

锁定粒度最小，发生锁冲突
的概率最低，并发度也最高

- 封锁类型

封锁分为排它锁和共享锁两种。排它锁，简称X锁，又称独占锁或写锁；共享锁，简称S锁，又称读锁。



一个事务对数据对象A加了X锁，那么该事务可以对A进行读和写，但其加锁期间其他事务不能对A加任何锁，直到X锁释放。

一个事务对数据对象A加了S锁，那么该事务可以对A进行读操作，但不能进行写操作，同时在其加锁期间其他事务能对A加S锁，但不能加X锁，直到S锁释放。

封锁和解锁的语法格式如下：

```
LOCK TABLES tbl_name {READ | WRITE}, [tbl_name {READ | WRITE}, ...]
```

```
UNLOCK TABLES
```

【例9-5】事务T₁获得对数据表account的排它锁权限，事务T₂尝试读取数据表中的数据。

T₁:

```
select * from account;
```

```
-- 为数据表account加上排它锁，转向事务T2
```

```
lock tables account write;
```

T₂:

```
--事务T2读取数据受阻
```

```
select * from account;
```

```
select * from account LIMIT 0, 1000
```

排它锁确保不会同时对同一资源进行多重更新。

【例9-6】事务T1获得对数据表account的共享锁权限的情况下：事务T2尝试读取、修改数据表中的数据；事务T2尝试对数据表继续添加共享锁；事务T2尝试对数据表继续添加排它锁。

T₁:

-- 为数据表account添加共享锁
lock tables account read;
select * from account;

id	R
account1	500
account2	900
account3	900

-- 修改account中的数据,执行不成功, 转向事务T₂

update account set R=800 where id="account1";

update account set R=800 where id="account1" Error Code: 1099. Table 'account' was locked with a READ lock and can't be updated

T₂:

-- 查询表中的数据, 返回结果, 可以读取
select * from account;

id	R
account1	500
account2	900
account3	900

【例9-6】事务T1获得对数据表account的共享锁权限的情况下：事务T2尝试读取、修改数据表中的数据；事务T2尝试对数据表继续添加共享锁；事务T2尝试对数据表继续添加排它锁。

T₂:

```
-- 修改表中的数据，执行不成功
update account set R=600 where id="account2";
-- 为数据表添加排它锁，执行不成功
lock tables account write;
-- 为数据表添加共享锁，执行成功
lock tables account read;
```

T₁:

```
-- 解锁
unlock tables
```

共享锁允许并发事务读取同一个资源。资源上存在共享锁时，任何其他事务都不能修改数据。除非将事务隔离级别设置为可重复读或更高级别，或者在事务生存周期内用锁定提示保留共享锁，那么一旦读取数据，便立即释放资源上的共享锁。

- 封锁协议

协议 在封锁时约定的一些规则，例如何时申请封锁、申请何种锁、持锁时间、何时释放等，被称为封锁协议 (Locking Protocol) 。



一级封锁协议

丢失更新

二级封锁协议

读脏数据

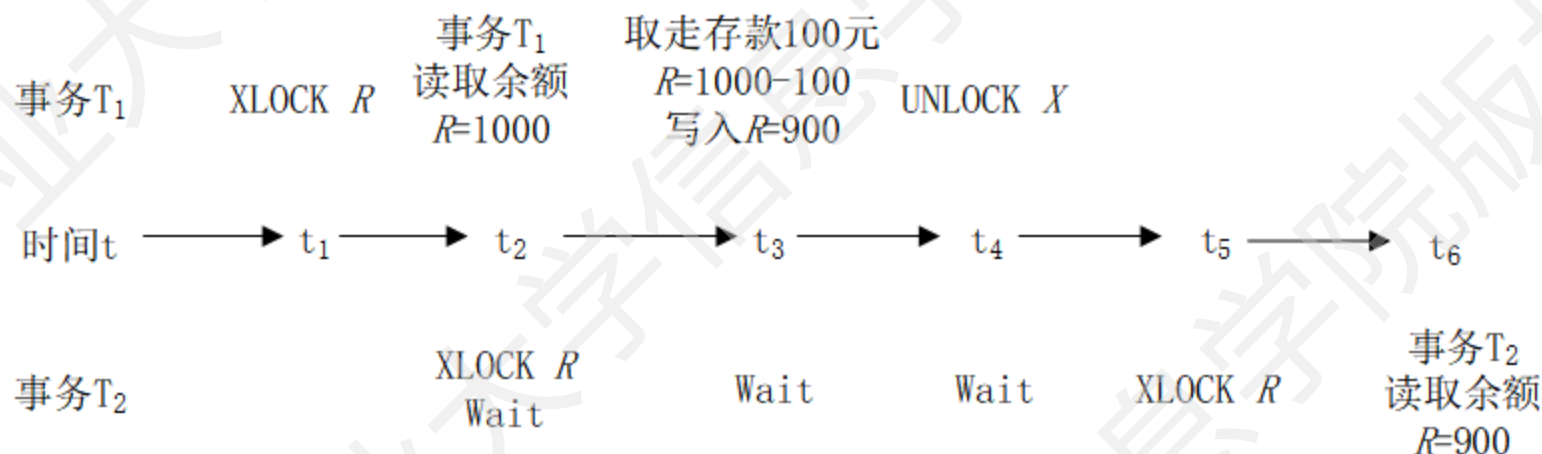
三级封锁协议

不可重复读

对封锁方式规定不同的规则，就形成了各种不同的封锁协议。

一级封锁协议：

事务T在修改数据A时必须先对其加X锁，直到事务结束才能释放锁。



- ✓ 在修改数据时对其加了X锁，不允许其他事务同时对数据进行修改，可以解决丢失更新问题。
- ✓ 事务结束包括正常结束（COMMIT）和非正常结束（ROLLBACK），保证事务T是可恢复的。
- ✓ 如果仅是读数据不对其进行修改，是不需要加锁的，不能保证可重复读和不读脏数据。

对封锁方式规定不同的规则，就形成了各种不同的封锁协议。

二级封锁协议：

在一级封锁协议的基础上，加上事务T在读取数据R之前必须先对其加S锁，读完后释放S锁。

- ✓ 防止了丢失更新，还可以进一步防止读脏数据；
- ✓ 由于读完数据后即可释放S锁，所以不能保证可重复读。

时 间	事务T1	数据库中R的值	事务T2
t0	XLOCK R	1000	
t1	FIND R		
t2	R=R-100		
t3	UPDATE R		
t4		900	SLOCK R
t5	ROLLBACK		WAIT
t6	UNLOCK X	1000	SLOCK R
t7			FIND R
t8			UNLOCK S

对封锁方式规定不同的规则，就形成了各种不同的封锁协议。

三级封锁协议：

在一级封锁协议的基础上，加上事务T在读取数据R之前必须先对其加S锁，读完后并不释放S锁，而直到事务T结束才释放。

- ✓ 事务T1在读R之前，先对R加S锁，这样其它事务只能再对R加S锁，而不能加X锁，即其他事务只能读R，而不能修改它们。可以防止丢失更新和不读脏数据。
- ✓ T1为验算再读R，这时读出的R仍然是原来读出的数值，即可重复读。

时 间	事务T1	数据库中R的值	事务T2
t0		1000	
t1	SLOCK R		
t2	FIND R		
t3			XLOCK R
t4	COMMIT		WAIT
t5	UNLOCK S		WAIT
t6			XLOCK R
t7			FIND R
t8			R=R-200
t9			UPDATE R
t10		800	UNLOCK X

- 死锁与活锁

死锁 (Dead Lock) 是指两个或更多的事务**同时处于等待状态**，每个事务都在等待其中另一个事务解除封锁，它才能继续执行下去，结果造成任何一个事务都无法继续执行。

时 间	事务 T_1	事务 T_2
t_0	封锁学生表	
t_1		封锁成绩表
t_2	要求封锁成绩表，等待	
t_3		要求封锁学生表，等待
t_4	等待	等待

- 死锁与活锁

死锁发生的必要条件包括：

互斥条件：事务在某一时间内独占资源，其他事务无法对资源进行操作。

请求与保持条件：事务因请求资源而阻塞时，对已获得的资源保持不放，导致该事务与其他事务都无法继续执行。

不剥夺条件：事务在获得资源后，如若事务没有解锁，则其他事务不能强行剥夺。

循环等待条件：多个事务之间形成一种头尾相接的循环等待资源关系，互相牵制。

避免死锁的常用方法：

①不同的事务同时并发存取多个表时，应尽量约定以相同的顺序访问各表，可以降低产生死锁的概率，通常称这种方法为顺序加锁法；

②事务如需要更新记录，应该直接申请足够级别的锁，即排它锁，而不应先申请共享锁。因为当事务申请排它锁时，数据已经被其他事务加上共享锁，那么可能造成锁冲突，甚至死锁；

③同一事务的执行如果需要多个数据对象，可以对这些数据对象一次性全部加锁，避免出现“请求与保持”，通常称这种方法为一次加锁法。

- 死锁与活锁

判断死锁的方法通常采用超时法和等待图法。

超时法

当两个事务互相等待的时间超过设置的某一阈值时，则判断形成了死锁，此时对其中一个事务进行回滚，则另一个等待的事务就能继续进行。回滚此事务相对于其他事务所占的时间可能会更多。

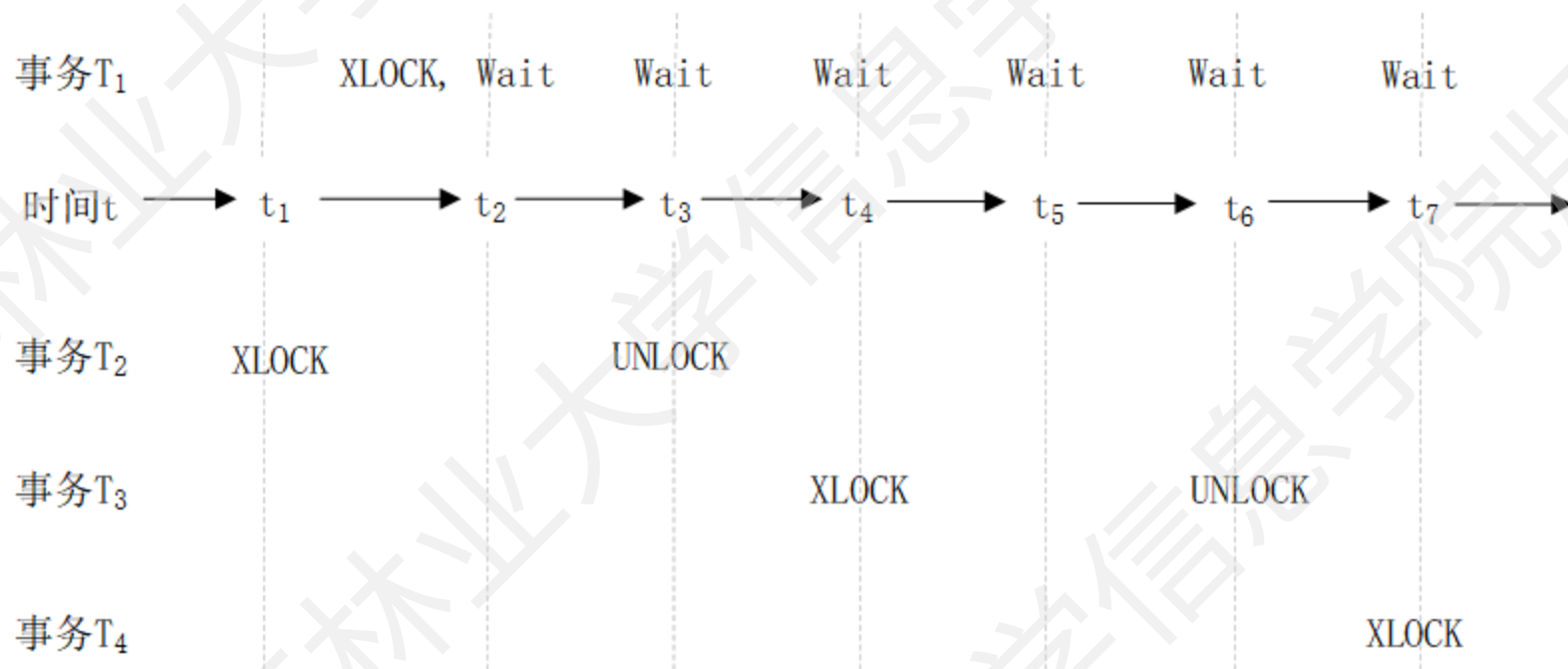


等待图 (wait-for graph) 法

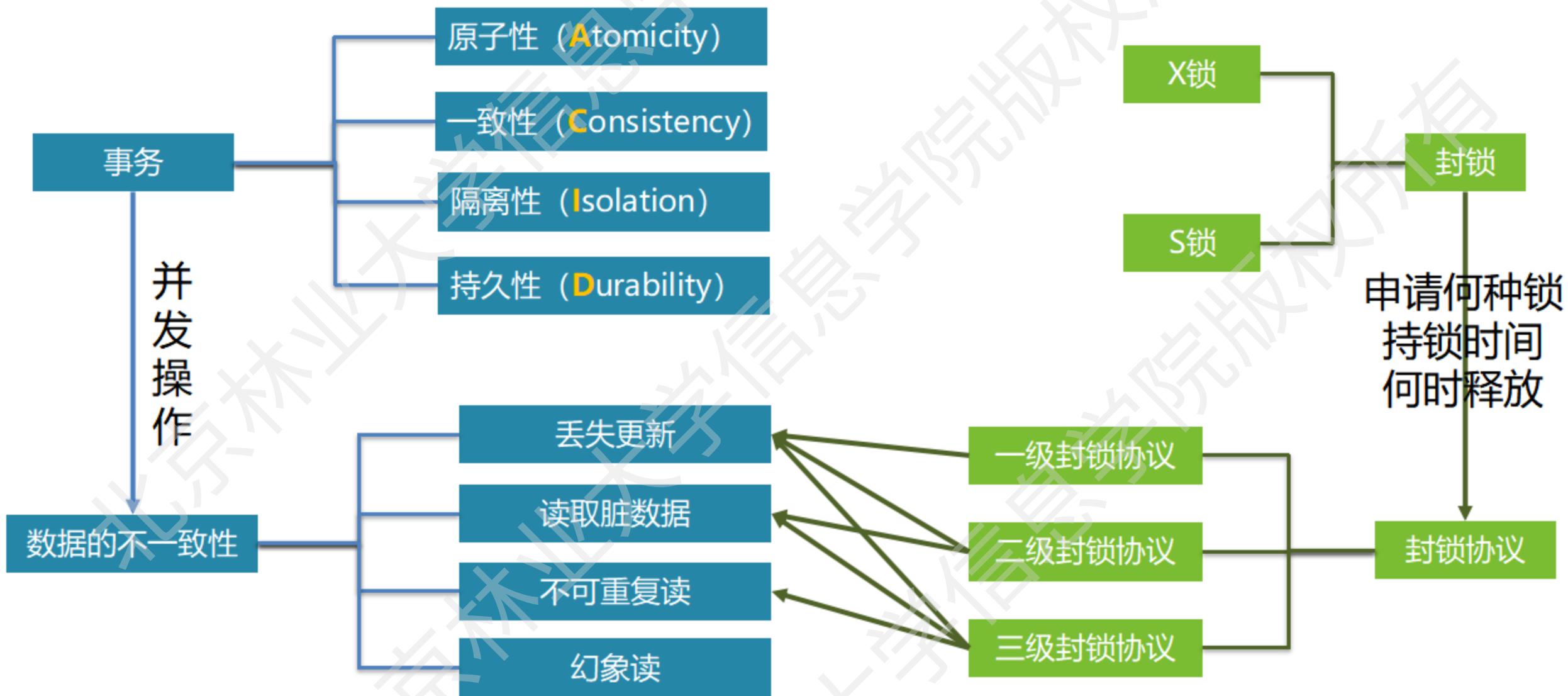
是一种更为主动的死锁检测方法。在每个事务请求锁并发生等待时都会判断是否存在回路，如果存在则有死锁，选择回滚操作最少的事务来破坏回路，从而解除死锁。

● 死锁与活锁

活锁 (Live Lock) 是指由于其他事务的封锁操作使某个事务永远处于等待状态, 得不到继续操作的机会。



活锁避免的方法就是采用先来先服务的策略, 按照请求封锁的次序对事务进行排队。





人邮教育

www.ryjiaoyu.com

更多样书申请和资源下载需求，请登录人邮教育社区
(www.ryjiaoyu.com)

海量图书方便查询

囊括各大品类，您想要的应有尽有

免费申请样书

教师免费申请样书，
我们将安排快递迅速送达

下载配套资源

教学视频、PPT课件、教学案例、
习题答案、模拟试卷等丰富资源
免费下载



优惠购书

教师可以申请最低折扣
学生直接优惠购买图书

成为作者

欢迎写文章 / 投稿，我们强大的编辑团队将为您提供专业和高效的编辑出版服务